Mini Assessment

Robust API Error Handling and Response Validation

Name: DASARI RAM MOHAN

Registration No: 99230040291

Department: CSE

College: Kalasalingam University of Research and Education

Q1. What should an application do when an API returns HTTP 500?

Answer: HTTP 500 indicates an internal server error on the API side. The application should treat the response as a temporary or server-side failure rather than assuming the requested operation succeeded. It should record useful diagnostic information, show a suitable message to the user, and retry only when retrying is appropriate. Repeated retries should use a controlled delay and a maximum retry limit so that the client does not overload an already failing service.

Q2. Why should an API client check the status code before processing the response body?

Answer: A status code tells the client whether the request was successful, rejected, redirected, or affected by a server problem. Processing the response body as if it were a successful result can produce incorrect data or unexpected exceptions. Checking the status first allows the program to choose the correct handling path, such as parsing success data, reporting an authentication error, or handling a server failure.

Q3. What is the purpose of a timeout when making an API request?

Answer: A timeout prevents a program from waiting indefinitely for a server that is slow, unavailable, or unreachable. By setting a reasonable timeout, the application can regain control and handle the failure gracefully. The program may report the issue, retry according to its retry policy, or continue with another operation. Timeouts are therefore important for application responsiveness and reliability.

Q4. How should an application handle a missing field in a successful JSON response?

Answer: The program should not assume that every optional field will always be present. It can use safe access such as `data.get('field')` and provide a suitable default when necessary. Required fields should be validated explicitly, while optional fields can be handled without terminating the entire program. This makes the application more tolerant of incomplete API responses.

Q5. Why is logging important when an API request fails?

Answer: Logging creates a record of what happened during a failed request. Useful information can include the endpoint, HTTP status, error type, timestamp, and a safe description of the failure. Logs help developers diagnose recurring problems and understand whether failures are caused by authentication, rate limits, network issues, invalid requests, or server errors. Sensitive information such as passwords or access tokens should not be logged.

Summary of Good API Practices

Practice	Purpose
Check HTTP status	Choose the correct success or error handling path.
Use timeouts	Prevent indefinite waiting for an API response.
Retry carefully	Recover from suitable temporary failures without overloading the server.
Validate JSON fields	Avoid crashes when optional or unexpected data is missing.

Practice	Purpose
Log failures safely	Support debugging while avoiding exposure of sensitive information.

Conclusion: Reliable API programs should expect failures and unexpected responses. Status-code checks, timeouts, controlled retries, safe JSON access, and meaningful logging help create applications that remain stable even when external services do not respond as expected.